

# Hướng Dẫn Hoàn Thành Assignment 02

Databricks + Delta Lake Lakehouse cho ABC Retail | Cập nhật: 2026-06-08

Mục tiêu: hoàn thành bài theo đúng README: sinh Landing data, xây Bronze append-only, quarantine lỗi, Silver chuẩn hóa/upsert/SCD2, Gold marts phục vụ BI, audit, time travel, CDF hoặc watermark, và tối ưu Delta.

## 1. Kết Quả Cần Nộp

Nên nộp một bộ artefact có thể chạy lại và giải thích được. Tối thiểu gồm notebook/script pipeline, dữ liệu landing giả lập, bảng Delta kết quả, vài query kiểm chứng và phần mô tả kiến trúc.

| Artefact       | Nội dung cần có                                                                        |
|----------------|----------------------------------------------------------------------------------------|
| Data generator | scripts/generate_mock_landing_data.py và requirements.txt.                             |
| Landing data   | clickstream_events, orders, customers, dim_fx_rates theo load_date/hour/batch_id.      |
| Bronze         | bronze_clickstream, bronze_orders, bronze_customers; append-only; có metadata lineage. |
| Quarantine     | Bảng lỗi theo source, lưu raw_payload, reason, source_file, batch_id, ingest_ts.       |
| Silver         | silver_events, silver_orders, silver_order_items, silver_customers.                    |
| Gold           | gold_daily_funnel, gold_revenue_daily, gold_customer_360, gold_data_quality_dashboard. |
| Delta demo     | MERGE, Time Travel, OPTIMIZE/ZORDER/VACUUM, CDF hoặc watermark.                        |

## 2. Quyết Định Thiết Kế Quan Trọng

- Bronze append-only: không update/delete dữ liệu raw đã ingest; metadata được thêm ở lúc ingest.
- Idempotency: dùng checkpoint Auto Loader/file tracking ở Bronze, deduplicate/MERGE deterministic ở Silver, MERGE hoặc replace affected partitions ở Gold.
- Late data: clickstream cho phép đến trễ 7 ngày; Gold theo ngày cần recompute các ngày bị ảnh hưởng hoặc dùng CDF/watermark lookback.
- Schema drift: clickstream dùng Auto Loader schema evolution và rescued data; Silver chuẩn hóa device về struct.
- Customers SCD2: dùng user\_id làm natural key, hash các cột thay đổi, effective\_from/effective\_to/is\_current.
- Orders: silver\_orders giữ latest state SCD1; silver\_order\_items nên version theo order\_id, sku, order\_updated\_at để bảo toàn lịch sử update item.

## 3. Chuẩn Bị Data Và Môi Trường

### 3.1. Sinh Landing Data Local

Trong workspace hiện tại đã có script sinh dữ liệu. Nếu cần sinh lại volume lớn, chạy:

```
myenv/bin/pip install -r requirements.txt
```

```
myenv/bin/python scripts/generate_mock_landing_data.py --output landing --overwrite --  
users 20000 --skus 1000 --clickstream-events 500000 --order-updates 80000 --  
customer-versions 30000
```

Data sinh ra có nhiều tình huống cần cho bài: duplicate event\_id, late records, anonymous user, schema drift ở device, malformed JSON, missing key, invalid timestamp, order status timeline, customer SCD2, stale update và FX rate missing.

### 3.2. Hiểu Đúng Unity Catalog Volume

Volume là một object được Unity Catalog quản trị dành cho file-based data như JSON, CSV, Parquet, checkpoint và schema metadata. Với path dưới đây, abc\_retail là catalog, dev là schema, landing là tên volume; phần sau landing mới là thư mục/file bên trong volume.

```
# Cú pháp path chuẩn  
/Volumes/<catalog>/<schema>/<volume>/<path>/<file>
```

```
# Ví dụ  
/Volumes/abc_retail/dev/landing/  
clickstream_events/  
orders/  
customers/  
dim_fx_rates/
```

Không tạo ba thư mục đầu bằng mkdir. /Volumes/abc\_retail/dev/landing chỉ tồn tại sau khi catalog abc\_retail, schema dev và volume landing đã được tạo trong Unity Catalog. Volume cần Unity Catalog-enabled compute và Databricks Runtime 13.3 LTS trở lên.

### 3.3. Chọn Managed Hay External Volume

| Loại            | Khi nên dùng                                                              | Đặc điểm                                                                         |
|-----------------|---------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Managed volume  | Assignment, demo, file chỉ được quản lý trong Databricks.                 | Databricks quản lý vị trí và vòng đời storage. Tạo đơn giản, không cần LOCATION. |
| External volume | Production landing khi upstream ghi trực tiếp vào bucket/container cloud. | Unity Catalog quản trị quyền, nhưng bạn quản lý cloud path và vòng đời file.     |

Cho bài này, dùng managed volume là đủ và ít cấu hình nhất. Nên tách hai volume: landing chỉ chứa input raw; ops chứa checkpoint và schema metadata của Auto Loader. Tách như vậy giúp phân quyền và tránh người ingest vô tình sửa checkpoint.

### 3.4. Tạo Catalog, Schema Và Volumes

Nếu account báo “Metastore storage root URL does not exist” và “Default Storage is enabled”: đừng tự điền một S3/ADLS/GCS path ngẫu nhiên. Trong Catalog Explorer, chọn Create catalog, nhập abc\_retail, chọn Use default storage, rồi Create. Sau đó dùng serverless notebook hoặc serverless SQL warehouse để thao tác với catalog này. Default storage không dùng được từ classic compute.

Kiểm tra catalog hiện có trước. Nếu abc\_retail chưa tồn tại, cách khuyến nghị cho account dùng Default Storage là tạo bằng UI: Catalog -> Create catalog -> Catalog name = abc\_retail -> Use default storage -> Create.

```
-- Chạy trên serverless compute sau khi tạo catalog bằng UI
SHOW CATALOGS;
DESCRIBE CATALOG EXTENDED abc_retail;

CREATE SCHEMA IF NOT EXISTS abc_retail.dev;

CREATE VOLUME IF NOT EXISTS abc_retail.dev.landing
COMMENT 'Raw hourly files uploaded by upstream';

CREATE VOLUME IF NOT EXISTS abc_retail.dev.ops
COMMENT 'Auto Loader checkpoints and inferred schemas';

SHOW VOLUMES IN abc_retail.dev;
DESCRIBE VOLUME abc_retail.dev.landing;
```

Nếu đang ở một serverless workspace và có quyền CREATE CATALOG, SQL CREATE CATALOG có thể dùng Default Storage mà không cần MANAGED LOCATION. Nếu lệnh vẫn báo storage root như trên, dùng UI để tạo catalog và kiểm tra lại compute đang là serverless.

```
-- Chạy trên serverless compute phù hợp
CREATE CATALOG IF NOT EXISTS abc_retail
COMMENT 'ABC Retail assignment catalog';
```

Một lựa chọn khác là dùng workspace catalog đã có managed storage. Chạy SHOW CATALOGS, chọn catalog của workspace mà bạn có quyền, rồi thay abc\_retail trong toàn bộ path/config bằng tên catalog đó.

Nếu cần external volume, cloud path phải là một subdirectory hợp lệ trong external location và không overlap bằng/volume khác:

```
CREATE EXTERNAL VOLUME abc_retail.dev.landing_external
LOCATION 's3://my-bucket/abc-retail/landing';
```

### 3.5. Cấp Quyền

Một principal muốn đọc file cần USE CATALOG, USE SCHEMA và READ VOLUME. Muốn upload, xóa hoặc sửa file cần thêm WRITE VOLUME. Tên data\_engineers bên dưới chỉ là ví dụ và group đó phải thực sự tồn tại trong Databricks account trước khi GRANT.

Assignment cá nhân: nếu chính bạn tạo catalog/schema/volume thì bạn là owner và thường không cần chạy GRANT. Nếu muốn demo GRANT, chạy SELECT session\_user() để lấy đúng username/email rồi thay vào principal. Không dùng CREATE GROUP trong SQL để sửa lỗi này vì lệnh đó tạo workspace-local group, không tương thích với Unity Catalog.

```
-- Xác định principal hiện tại
SELECT session_user();

-- Ví dụ kết quả: student@example.com
GRANT USE CATALOG ON CATALOG abc_retail TO `student@example.com`;
```

```
GRANT USE SCHEMA ON SCHEMA abc_retail.dev TO `student@example.com`;
GRANT READ VOLUME, WRITE VOLUME
ON VOLUME abc_retail.dev.landing TO `student@example.com`;
```

Trong môi trường team/production, tạo account-level group `data_engineers` bằng Admin Settings/Account Console hoặc Account Groups API/CLI, thêm thành viên và gán group vào workspace. Sau đó mới dùng:

```
GRANT USE CATALOG ON CATALOG abc_retail TO `data_engineers`;
GRANT USE SCHEMA ON SCHEMA abc_retail.dev TO `data_engineers`;
```

```
GRANT READ VOLUME, WRITE VOLUME
ON VOLUME abc_retail.dev.landing TO `data_engineers`;
```

```
GRANT READ VOLUME, WRITE VOLUME
ON VOLUME abc_retail.dev.ops TO `data_engineers`;
```

Có thể dùng system group `'account users'` cho demo, nhưng quyền này áp dụng cho toàn bộ user và service principal trong Databricks account nên không khuyến nghị nếu account dùng chung.

### 3.6. Upload Landing Data

- Không dùng màn hình Azure Portal Upload blob cho nguyên folder: dialog này chủ yếu chọn file, không upload recursive cả cây thư mục landing.
- Python uploader trong workspace: `scripts/upload_landing_to_azure.py` upload recursive, hỗ trợ dry-run, SAS/Azure Identity, concurrency và verify.
- Azure Storage Explorer: dùng Upload Folder để upload nguyên thư mục landing và giữ cấu trúc con.
- AzCopy: lựa chọn tốt nhất cho nhiều file; dùng `--recursive=true` để copy cả cây thư mục.
- Databricks CLI: dùng `databricks fs cp`. Với CLI, volume path phải bắt đầu bằng `dbfs:/Volumes`.
- Giữ nguyên cấu trúc `load_date/hour/batch_id` vì pipeline sẽ parse lineage từ path.

```
# Preview, không upload
myenv/bin/python scripts/upload_landing_to_azure.py --source landing --account-name
<storage-account> --container <container> --prefix landing --dry-run

# Upload bằng SAS token lưu trong environment và verify kết quả
export AZURE_STORAGE_SAS_TOKEN='<sas-token>'
myenv/bin/python scripts/upload_landing_to_azure.py --source landing --account-name
<storage-account> --container <container> --prefix landing --workers 16 --verify

# AzCopy: source là folder landing, destination là container root.
# Mắc mặc định --as-subdir=true nên kết quả là container/landing/...
azcopy copy "landing" "https://<storage-account>.blob.core.windows.net/abc-retail?<sas-token>" --recursive=true --overwrite=true

# Nếu dùng Entra ID thay vì SAS
azcopy login
azcopy copy "landing" "https://<storage-account>.blob.core.windows.net/abc-retail" --recursive=true --overwrite=true

# Hoàn thành copy nội dung bên trong landing vào prefix landing:
azcopy copy "landing/*" "https://<storage-account>.blob.core.windows.net/abc-retail/landing?<sas-token>" --recursive=true --overwrite=true

databricks fs cp -r landing/clickstream_events dbfs:/Volumes/abc_retail/dev/landing/clickstream_events --overwrite

databricks fs cp -r landing/orders dbfs:/Volumes/abc_retail/dev/landing/orders --overwrite
```

```
databricks fs cp -r landing/customers dbfs:/Volumes/abc_retail/dev/landing/customers --
overwrite
```

```
databricks fs cp -r landing/dim_fx_rates dbfs:/Volumes/abc_retail/dev/landing/
dim_fx_rates --overwrite
```

### 3.7. Kiểm Tra Sau Upload

```
# Notebook Python
display(dbutils.fs.ls("/Volumes/abc_retail/dev/landing"))
display(dbutils.fs.ls("/Volumes/abc_retail/dev/landing/clickstream_events"))

# Spark ■■■c th■ m■t batch
sample = spark.read.json(
  "/Volumes/abc_retail/dev/landing/clickstream_events/load_date=2026-05-01/hour=00/*"
)
sample.printSchema()
display(sample.limit(10))

-- SQL: li■■t kê file trong volume
LIST '/Volumes/abc_retail/dev/landing/clickstream_events';
```

### 3.8. Vì Sao Volume Dễ Audit Hơn

- Namespace rõ: path luôn chỉ ra catalog/schema/volume, thay vì một mount path khó biết owner.
- Phân quyền tập trung: READ VOLUME và WRITE VOLUME được quản trị trong Unity Catalog.
- Hoạt động được ghi audit: Unity Catalog và filesystem operations có thể xuất hiện trong system.access.audit nếu workspace/account đã bật system tables.
- Path lineage ổn định: source\_file lưu trong Bronze chỉ thẳng đến file nằm trong volume và batch folder.
- Tách trách nhiệm: upstream writer có thể chỉ ghi landing; pipeline reader chỉ đọc landing và ghi checkpoint vào ops.

Giới hạn cần hiểu: Volume không tự tạo record-level lineage. Bạn vẫn phải thêm source\_file, batch\_id, load\_date và ingest\_ts vào Bronze, đồng thời ghi audit\_pipeline\_runs. Volume cung cấp governance và audit ở cấp object/file access; metadata Bronze cung cấp lineage đến từng record.

### 3.9. Cấu Hình Notebook Đầu Tiên

```
spark.conf.set("spark.sql.session.timeZone", "UTC")

catalog = "abc_retail"
schema = "dev"
landing_path = "/Volumes/abc_retail/dev/landing"
checkpoint_path = "/Volumes/abc_retail/dev/ops/checkpoints"

spark.sql(f"USE CATALOG {catalog}")
spark.sql(f"USE SCHEMA {schema}")
```

## 4. Tạo Bảng Control, Audit Và Quarantine

Trước khi ingest dữ liệu, tạo bảng audit và quarantine. Đây là phần giúp bài làm vượt khỏi ETL cơ bản: mỗi batch biết đọc bao nhiêu dòng, ghi bao nhiêu dòng, lỗi bao nhiêu dòng và đến từ file nào.

```

CREATE TABLE IF NOT EXISTS audit_pipeline_runs (
  pipeline_name STRING,
  batch_id STRING,
  start_ts TIMESTAMP,
  end_ts TIMESTAMP,
  status STRING,
  rows_read BIGINT,
  rows_written BIGINT,
  rows_quarantined BIGINT,
  source_paths ARRAY<STRING>,
  notebook_job_run_id STRING,
  error_message STRING
) USING DELTA;

CREATE TABLE IF NOT EXISTS quarantine_clickstream (
  source STRING, batch_id STRING, source_file STRING, ingest_ts TIMESTAMP,
  raw_payload STRING, reason STRING, error_category STRING
) USING DELTA;

CREATE TABLE IF NOT EXISTS quarantine_orders (
  source STRING, batch_id STRING, source_file STRING, ingest_ts TIMESTAMP,
  raw_payload STRING, reason STRING, error_category STRING
) USING DELTA;

CREATE TABLE IF NOT EXISTS quarantine_customers (
  source STRING, batch_id STRING, source_file STRING, ingest_ts TIMESTAMP,
  raw_payload STRING, reason STRING, error_category STRING
) USING DELTA;

CREATE TABLE IF NOT EXISTS schema_change_log (
  source STRING, observed_ts TIMESTAMP, schema_version STRING,
  source_file STRING, schema_json STRING
) USING DELTA;

```

Lưu ý: metadata như source\_file, batch\_id, load\_date nên được thêm ở Bronze bằng input\_file\_name() và parse path, không nên nhét sẵn vào dữ liệu giả lập. Như vậy lineage mới phản ánh đúng quá trình ingest.

## 5. Bronze Layer - Append Only

### 5.1. Pattern Metadata Chung

Dùng các helper column này cho mọi source:

```

from pyspark.sql import functions as F

def add_lineage_cols(df):
    source_file = F.input_file_name()
    return (
        df.withColumn("ingest_ts", F.current_timestamp())
        .withColumn("source_file", source_file)
        .withColumn("load_date", F.regexp_extract(source_file, r"load_date=([^\s/]+)", 1))
        .withColumn("batch_id", F.regexp_extract(source_file, r"batch_id=([^\s/]+)", 1))
    )

```

### 5.2. Bronze Clickstream

Clickstream có malformed JSON và schema drift. Cách an toàn là dùng Auto Loader JSON với rescued data, schema evolution và schemaHints cho device dạng struct. Dữ liệu lỗi parse/type mismatch đi vào \_rescued\_data để Silver/quarantine xử lý.

```

bronze_clickstream_df = (

```

```

spark.readStream.format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("cloudFiles.schemaLocation", f"{checkpoint_path}/schema/clickstream")
  .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
  .option("rescuedDataColumn", "_rescued_data")
  .option("cloudFiles.schemaHints", "device STRUCT<os:STRING,model:STRING>, attributes
MAP<STRING,STRING>")
  .option("pathGlobFilter", "*.json")
  .load(f"{landing_path}/clickstream_events")
)

bronze_clickstream_df = add_lineage_cols(
  bronze_clickstream_df.withColumn("raw_payload", F.to_json(F.struct("*")))
)

(bronze_clickstream_df.writeStream
  .format("delta")
  .option("checkpointLocation", f"{checkpoint_path}/bronze_clickstream")
  .trigger(availableNow=True)
  .toTable("bronze_clickstream"))

```

Sau mỗi run, lấy schema JSON của bronze\_clickstream, hash nó và chỉ append vào schema\_change\_log khi hash khác version gần nhất. Đây là bằng chứng schema drift/evolution đã được quan sát.

```

import hashlib

schema_json = spark.table("bronze_clickstream").schema.json()
schema_version = hashlib.sha256(schema_json.encode("utf-8")).hexdigest()

# So sánh schema_version với bản ghi gần nhất rồi append nếu thay đổi.

```

### 5.3. Bronze Orders Và Customers

Data generator có orders dạng JSON/Parquet và customers dạng CSV/JSON. Không đọc mixed format trong một stream duy nhất; tạo loader riêng theo format nhưng append vào cùng bảng Bronze.

```

# Orders JSON
orders_json = spark.readStream.format("cloudFiles") \
  .option("cloudFiles.format", "json") \
  .option("cloudFiles.schemaLocation", f"{checkpoint_path}/schema/orders_json") \
  .option("rescuedDataColumn", "_rescued_data") \
  .option("pathGlobFilter", "*.json") \
  .load(f"{landing_path}/orders")

add_lineage_cols(orders_json.withColumn("raw_payload", F.to_json(F.struct("*")))) \
  .writeStream.format("delta") \
  .option("checkpointLocation", f"{checkpoint_path}/bronze_orders_json") \
  .trigger(availableNow=True).toTable("bronze_orders")

# Orders Parquet
orders_parquet = spark.readStream.format("cloudFiles") \
  .option("cloudFiles.format", "parquet") \
  .option("cloudFiles.schemaLocation", f"{checkpoint_path}/schema/orders_parquet") \
  .option("pathGlobFilter", "*.parquet") \
  .load(f"{landing_path}/orders")

add_lineage_cols(orders_parquet.withColumn("raw_payload", F.to_json(F.struct("*")))) \
  .writeStream.format("delta") \
  .option("checkpointLocation", f"{checkpoint_path}/bronze_orders_parquet") \
  .trigger(availableNow=True).toTable("bronze_orders")

```

Tương tự cho customers: một loader CSV, một loader JSON, cùng append vào bronze\_customers.

```

customers_csv = spark.readStream.format("cloudFiles") \
  .option("cloudFiles.format", "csv") \
  .option("header", "true") \
  .option("cloudFiles.schemaLocation", f"{checkpoint_path}/schema/customers_csv") \
  .option("pathGlobFilter", "*.csv") \
  .load(f"{landing_path}/customers")

```

```
customers_json = spark.readStream.format("cloudFiles").option("cloudFiles.format",
"json").option("cloudFiles.schemaLocation", f"{checkpoint_path}/schema/customers_json")
.option("pathGlobFilter", "*.json").load(f"{landing_path}/customers")
```

## 6. Silver Events

Silver Events phải chuẩn hóa timestamp UTC, chuẩn hóa device, deduplicate theo event\_id và quarantine record lỗi.

```
from pyspark.sql import Window
from pyspark.sql import functions as F

src = spark.table("bronze_clickstream")

device_from_rescue = F.get_json_object("_rescued_data", "$.device")
valid_rescued_device = device_from_rescue.rlike(r"^[^|]+\|[^|]+$")
normalized = (
    src.withColumn("event_ts", F.expr("try_to_timestamp(event_time)"))
        .withColumn(
            "device_norm",
            F.when(F.col("device.os").isNotNull(),
                F.struct(F.col("device.os").alias("os"), F.col("device.model").alias(
                    "model"))))
            .when(valid_rescued_device,
                F.struct(F.split(device_from_rescue, "\\|")[0].alias("os"),
                    F.split(device_from_rescue, "\\|")[1].alias("model")))
        )
        .withColumn("event_date", F.to_date("event_ts"))
)

bad = normalized.where(
    F.col("event_id").isNull() |
    F.col("event_ts").isNull() |
    F.col("device_norm").isNull()
)

bad.selectExpr(
    "'clickstream' as source", "batch_id", "source_file", "ingest_ts",
    "raw_payload",
    """CASE
        WHEN event_id IS NULL THEN 'missing event_id'
        WHEN event_ts IS NULL THEN 'invalid event_time'
        WHEN device_norm IS NULL THEN 'invalid device/schema mismatch'
        ELSE 'unknown'
    END as reason""",
    """CASE
        WHEN event_id IS NULL THEN 'missing_key'
        WHEN event_ts IS NULL THEN 'invalid_timestamp'
        ELSE 'schema_mismatch'
    END as error_category"""
).write.mode("append").saveAsTable("quarantine_clickstream")

valid = normalized.where(
    F.col("event_id").isNotNull() &
    F.col("event_ts").isNotNull() &
    F.col("device_norm").isNotNull()
)

w = Window.partitionBy("event_id").orderBy(F.expr("try_to_timestamp(ingest_time)").
desc_nulls_last(), F.col("ingest_ts").desc())
ranked = valid.withColumn("rn", F.row_number().over(w))

# Ghi ranked.rn > 1 vào quarantine_clickstream và i error_category = duplicate_key.
dedup = ranked.where("rn = 1").drop("rn")
dedup.createOrReplaceTempView("stg_events")
```

Tạo bảng và MERGE deterministic để rerun không tạo trùng:

```

CREATE TABLE IF NOT EXISTS silver_events (
  event_id STRING NOT NULL,
  event_time TIMESTAMP NOT NULL,
  event_date DATE,
  ingest_time TIMESTAMP,
  user_id STRING,
  session_id STRING,
  event_type STRING,
  page STRING,
  device STRUCT<os:STRING,model:STRING>,
  attributes MAP<STRING,STRING>,
  source_file STRING,
  batch_id STRING,
  ingest_ts TIMESTAMP
) USING DELTA
PARTITIONED BY (event_date);

MERGE INTO silver_events t
USING (
  SELECT event_id, event_ts AS event_time, event_date,
    try_to_timestamp(ingest_time) AS ingest_time,
    user_id, session_id, event_type, page,
    device_norm AS device, attributes,
    source_file, batch_id, ingest_ts
  FROM stg_events
) s
ON t.event_id = s.event_id
WHEN MATCHED AND s.ingest_time >= t.ingest_time THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;

ALTER TABLE silver_events
ADD CONSTRAINT event_id_required CHECK (event_id IS NOT NULL);

ALTER TABLE silver_events
ADD CONSTRAINT event_time_required CHECK (event_time IS NOT NULL);

```

## 7. Silver Orders Và Items

silver\_orders giữ latest state theo order\_id + updated\_at. silver\_order\_items nên version theo order\_id, sku, order\_updated\_at để không mất lịch sử qty/price qua từng update.

### 7.1. Tạo dim\_fx\_rates

```

fx = spark.read.option("header", "true").csv(f"{landing_path}/dim_fx_rates/fx_rates.csv")
fx.selectExpr("to_date(rate_date) as rate_date", "currency", "cast(rate_to_usd as double)
as rate_to_usd").write.mode("overwrite").format("delta").saveAsTable("dim_fx_rates")

```

## 7.2. Validate, FX Conversion Và Quarantine

```
orders = (
  spark.table("bronze_orders")
    .withColumn("order_ts", F.expr("try_to_timestamp(order_time)"))
    .withColumn("updated_ts", F.expr("try_to_timestamp(updated_at)"))
    .withColumn("order_date", F.to_date("order_ts"))
)

exploded = orders.withColumn("item", F.explode_outer("items")).alias("o")
fx = spark.table("dim_fx_rates").alias("fx")
with_fx = exploded.join(
  fx,
  (F.col("o.order_date") == F.col("fx.rate_date")) &
  (F.col("o.currency") == F.col("fx.currency")),
  "left"
)

bad_orders = with_fx.where(
  F.col("order_id").isNull() |
  F.col("order_ts").isNull() |
  F.col("updated_ts").isNull() |
  F.col("item.sku").isNull() |
  (F.col("item.qty") <= 0) |
  (F.col("item.unit_price") < 0) |
  (F.col("rate_to_usd").isNull())
)
```

## 7.3. MERGE silver\_orders

```
CREATE TABLE IF NOT EXISTS silver_orders (
  order_id STRING NOT NULL,
  order_time TIMESTAMP,
  order_date DATE,
  user_id STRING,
  currency STRING,
  discount_code STRING,
  shipping_address STRUCT<line1:STRING,city:STRING,country:STRING,postal_code:STRING>,
  status STRING,
  updated_at TIMESTAMP,
  source_file STRING,
  batch_id STRING,
  ingest_ts TIMESTAMP
) USING DELTA
PARTITIONED BY (order_date);

MERGE INTO silver_orders t
USING stg_orders_latest s
ON t.order_id = s.order_id
WHEN MATCHED AND s.updated_at >= t.updated_at THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

## 7.4. MERGE silver\_order\_items Versioned

```
CREATE TABLE IF NOT EXISTS silver_order_items (  
  order_id STRING,  
  sku STRING,  
  order_updated_at TIMESTAMP,  
  qty INT,  
  unit_price_original DOUBLE,  
  currency STRING,  
  unit_price_usd DOUBLE,  
  line_amount_original DOUBLE,  
  line_amount_usd DOUBLE,  
  is_latest BOOLEAN,  
  source_file STRING,  
  batch_id STRING,  
  ingest_ts TIMESTAMP  
) USING DELTA;  
  
MERGE INTO silver_order_items t  
USING stg_order_items s  
ON t.order_id = s.order_id  
  AND t.sku = s.sku  
  AND t.order_updated_at = s.order_updated_at  
WHEN MATCHED THEN UPDATE SET *  
WHEN NOT MATCHED THEN INSERT *;
```

Sau MERGE, recompute `is\_latest` cho các `(order\_id, sku)` bị ảnh hưởng bằng window `order_updated_at desc`. Điều này giúp Gold lấy đúng item latest nhưng vẫn giữ lịch sử các lần update.

## 8. Silver Customers - SCD Type 2

Customer SCD2 là phần nên làm cẩn thận. Cách bền nhất là dedup incoming theo `user_id + updated_at`, tính hash các cột thay đổi, sau đó rebuild timeline cho các user bị ảnh hưởng. Late record cũ sẽ được chèn vào đúng khoảng `effective_from/effective_to` thay vì overwrite current.

```
CREATE TABLE IF NOT EXISTS silver_customers (  
  user_id STRING,  
  email STRING,  
  phone STRING,  
  country STRING,  
  created_at TIMESTAMP,  
  updated_at TIMESTAMP,  
  effective_from TIMESTAMP,  
  effective_to TIMESTAMP,  
  is_current BOOLEAN,  
  change_hash STRING,  
  source_file STRING,  
  batch_id STRING,  
  ingest_ts TIMESTAMP  
) USING DELTA;
```

Rule DQ nên rõ ràng: thiếu `user_id`, email invalid, phone null hoặc `updated_at` lỗi thì quarantine.

```
customers = spark.table("bronze_customers") \  
  .withColumn("created_ts", F.expr("try_to_timestamp(created_at)")) \  
  .withColumn("updated_ts", F.expr("try_to_timestamp(updated_at)"))  
  
valid_customers = customers.where(  
  F.col("user_id").isNotNull() &  
  F.col("updated_ts").isNotNull() &  
  F.col("email").rlike(r"^[^@\\s]+@[^@\\s]+\\.([^@\\s]+)$") &  
  F.col("phone").isNotNull()  
) .withColumn(  
  "change_hash", F.expr("hash(concat(email, phone, updated_ts))")  
)
```

```

    "change_hash",
    F.sha2(F.concat_ws("||", F.coalesce("email", F.lit("")), F.coalesce("phone", F.lit("")),
F.coalesce("country", F.lit("))), 256)
)

```

Tạo timeline cho các user bị ảnh hưởng. Bỏ qua record liên tiếp không đổi hash để SCD2 không phình vô ích:

```

from pyspark.sql import Window
from pyspark.sql import functions as F

affected_users = valid_customers.select("user_id").distinct()

existing = spark.table("silver_customers").join(affected_users, "user_id", "inner")
select("user_id", "email", "phone", "country", "created_at", "updated_at",
      "source_file", "batch_id", "ingest_ts", "change_hash")

incoming = valid_customers.selectExpr(
    "user_id", "email", "phone", "country",
    "created_ts as created_at", "updated_ts as updated_at",
    "source_file", "batch_id", "ingest_ts", "change_hash"
)

hist = existing.unionByName(incoming).dropDuplicates(["user_id", "updated_at",
"change_hash"])
w = Window.partitionBy("user_id").orderBy("updated_at")

timeline = (
    hist.withColumn("prev_hash", F.lag("change_hash").over(w))
        .where("prev_hash IS NULL OR prev_hash <> change_hash")
        .withColumn("effective_from", F.col("updated_at"))
        .withColumn("effective_to", F.lead("updated_at").over(w))
        .withColumn("is_current", F.col("effective_to").isNull())
        .drop("prev_hash")
)

timeline.createOrReplaceTempView("stg_customer_scd2")

```

Sau khi có `stg\_customer\_scd2`, dùng MERGE bắt buộc cho Silver Customers:

```

MERGE INTO silver_customers t
USING stg_customer_scd2 s
ON t.user_id = s.user_id AND t.effective_from = s.effective_from
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;

```

Điểm cần giải thích trong bài: record đến trễ nhưng updated\_at cũ hơn current không bị bỏ qua. Nó được đưa vào timeline nếu hợp lệ; nếu phá vỡ rule như trùng timestamp nhưng khác dữ liệu, đưa quarantine với category late\_conflict.

## 9. Gold Layer

### 9.1. gold\_daily\_funnel

Quy định anonymous: nếu user\_id null thì dùng actor\_id = concat('anon:', session\_id). Bảng theo event\_date và đếm unique actor theo từng bước.

```
CREATE OR REPLACE TABLE gold_daily_funnel AS
SELECT
    event_date,
    count(DISTINCT CASE WHEN event_type = 'view' THEN coalesce(user_id, concat('anon:',
session_id)) END) AS view_users,
    count(DISTINCT CASE WHEN event_type = 'add_to_cart' THEN coalesce(user_id, concat('anon:',
session_id)) END) AS add_to_cart_users,
    count(DISTINCT CASE WHEN event_type = 'purchase' THEN coalesce(user_id, concat('anon:',
session_id)) END) AS purchase_users,
    current_timestamp() AS updated_at
FROM silver_events
WHERE event_type IN ('view', 'add_to_cart', 'purchase')
GROUP BY event_date;
```

### 9.2. gold\_revenue\_daily

Net revenue lấy latest state. Cancelled/returned tính doanh thu 0. Country nên lấy từ customer current hoặc shipping\_address.country; nếu cần historical country thì temporal join SCD2 theo order\_time.

```
CREATE OR REPLACE TABLE gold_revenue_daily AS
WITH latest_items AS (
    SELECT * FROM silver_order_items WHERE is_latest = true
),
order_revenue AS (
    SELECT
        o.order_id,
        o.order_date,
        coalesce(c.country, o.shipping_address.country) AS country,
        CASE WHEN o.status IN ('cancelled', 'returned') THEN 0
            ELSE sum(i.line_amount_usd)
        END AS net_revenue_usd
    FROM silver_orders o
    JOIN latest_items i ON o.order_id = i.order_id
    LEFT JOIN silver_customers c ON o.user_id = c.user_id AND c.is_current = true
    GROUP BY o.order_id, o.order_date, coalesce(c.country, o.shipping_address.country), o.
status
)
SELECT order_date, country, sum(net_revenue_usd) AS net_revenue_usd, count(*) AS orders
FROM order_revenue
GROUP BY order_date, country;
```

### 9.3. gold\_customer\_360

```
CREATE OR REPLACE TABLE gold_customer_360 AS
WITH current_customers AS (
  SELECT * FROM silver_customers WHERE is_current = true
),
order_stats AS (
  SELECT user_id,
         count(DISTINCT order_id) AS total_orders,
         sum(CASE WHEN status NOT IN ('cancelled','returned') THEN 1 ELSE 0 END) AS
completed_orders,
         max(CASE WHEN status NOT IN ('cancelled','returned') THEN order_time END) AS
last_purchase_ts
  FROM silver_orders
 GROUP BY user_id
),
revenue_stats AS (
  SELECT o.user_id, sum(i.line_amount_usd) AS total_revenue_usd
  FROM silver_orders o
 JOIN silver_order_items i ON o.order_id = i.order_id AND i.is_latest = true
 WHERE o.status NOT IN ('cancelled','returned')
 GROUP BY o.user_id
),
device_rank AS (
  SELECT user_id, device.os AS os, device.model AS model,
         row_number() OVER (PARTITION BY user_id ORDER BY count(*) DESC) AS rn
  FROM silver_events
 WHERE user_id IS NOT NULL
 GROUP BY user_id, device.os, device.model
)
SELECT c.user_id, c.email, c.phone, c.country,
       coalesce(o.total_orders, 0) AS total_orders,
       coalesce(r.total_revenue_usd, 0) AS total_revenue_usd,
       o.last_purchase_ts,
       struct(d.os, d.model) AS most_common_device
  FROM current_customers c
 LEFT JOIN order_stats o ON c.user_id = o.user_id
 LEFT JOIN revenue_stats r ON c.user_id = r.user_id
 LEFT JOIN device_rank d ON c.user_id = d.user_id AND d.rn = 1;
```

### 9.4. gold\_data\_quality\_dashboard

```
CREATE OR REPLACE TABLE gold_data_quality_dashboard AS
SELECT load_date, source, error_category, count(*) AS error_rows, 0 AS valid_rows
FROM (
  SELECT to_date(ingest_ts) AS load_date, source, error_category FROM quarantine_clickstream
 UNION ALL
  SELECT to_date(ingest_ts) AS load_date, source, error_category FROM quarantine_orders
 UNION ALL
  SELECT to_date(ingest_ts) AS load_date, source, error_category FROM quarantine_customers
)
GROUP BY load_date, source, error_category
UNION ALL
SELECT to_date(end_ts) AS load_date, pipeline_name AS source, 'valid' AS error_category,
       0 AS error_rows, sum(rows_written) AS valid_rows
  FROM audit_pipeline_runs
 WHERE status = 'SUCCESS'
 GROUP BY to_date(end_ts), pipeline_name;
```

## 10. Incremental Gold Bằng CDF Hoặc Watermark

### 10.1. CDF cho silver\_orders

```
ALTER TABLE silver_orders SET TBLPROPERTIES (delta.enableChangeDataFeed = true);
```

```
CREATE TABLE IF NOT EXISTS pipeline_watermarks (  
  table_name STRING,  
  last_version BIGINT,  
  updated_at TIMESTAMP  
) USING DELTA;
```

```
-- Lấy các ngày order bị ảnh hưởng từ CDF  
SELECT DISTINCT order_date  
FROM table_changes('silver_orders', <last_version_plus_1>)  
WHERE _change_type IN ('insert', 'update_postimage', 'delete');
```

Sau đó chỉ recompute gold\_revenue\_daily cho danh sách order\_date bị ảnh hưởng. Với late data, ngày bị ảnh hưởng có thể nằm trong quá khứ; không dùng logic chỉ tính ngày hiện tại.

### 10.2. Nếu Workspace Không Hỗ Trợ CDF

```
-- Watermark có lookback 7 ngày để bắt late-arriving data  
WHERE ingest_ts > last_success_ingest_ts - INTERVAL 7 DAYS
```

Khi dùng watermark, luôn recompute các partition ngày nghiệp vụ nằm trong cửa sổ bị ảnh hưởng. Ví dụ clickstream late 7 ngày thì gold\_daily\_funnel nên replace các event\_date từ min(event\_date impacted) đến max(event\_date impacted).

## 11. Time Travel, Audit Và Idempotency Demo

### 11.1. Time Travel

```
DESCRIBE HISTORY silver_orders;
```

```
SELECT count(*) FROM silver_orders VERSION AS OF 3;
```

```
SELECT *  
FROM gold_revenue_daily TIMESTAMP AS OF '2026-06-08T10:00:00Z'  
WHERE order_date = '2026-05-01';
```

### 11.2. Idempotency Test

- 1 Chạy pipeline Bronze -> Silver -> Gold lần 1 và ghi lại count của từng bảng.
- 2 Chạy lại cùng input và cùng checkpoint. Bronze không đọc lại file; Silver/Gold không tăng duplicate.
- 3 Nếu reset checkpoint để demo replay, Silver vẫn không duplicate vì dedup event\_id và MERGE theo key.
- 4 Query kiểm chứng: count(distinct event\_id) = count(\*) trên silver\_events, order\_id unique trên silver\_orders current state.

```
SELECT count(*) AS rows, count(DISTINCT event_id) AS distinct_event_ids  
FROM silver_events;
```

```
SELECT order_id, count(*) c
FROM silver_orders
GROUP BY order_id
HAVING c > 1;
```

## 12. Optimization Và Vacuum

Chọn ít nhất hai bảng lớn: silver\_events và silver\_orders.

```
DESCRIBE DETAIL silver_events;
DESCRIBE DETAIL silver_orders;
```

```
OPTIMIZE silver_events ZORDER BY (event_id, user_id);
OPTIMIZE silver_orders ZORDER BY (order_id, user_id);
```

```
DESCRIBE HISTORY silver_events;
DESCRIBE DETAIL silver_events;
```

```
VACUUM silver_events RETAIN 720 HOURS;
VACUUM silver_orders RETAIN 720 HOURS;
```

- Partition silver\_events theo event\_date vì truy vấn funnel và late recompute theo ngày.
- ZORDER event\_id giúp dedup/lookups; user\_id giúp customer360 và phân tích hành vi.
- ZORDER order\_id giúp MERGE/order lookup; user\_id giúp join customer360.
- Retention 720 giờ tương đương 30 ngày: tốn storage hơn default 7 ngày nhưng phù hợp audit/time travel và late data.

Khi viết báo cáo, nhớ đưa chỉ số trước/sau: numFiles, sizeInBytes, avgFileSize = sizeInBytes / numFiles, và operationMetrics trong DESCRIBE HISTORY sau OPTIMIZE.

## 13. Thứ Tự Notebook Khuyến Nghị

| Notebook                   | Mục tiêu                                                          |
|----------------------------|-------------------------------------------------------------------|
| 00_config                  | Set catalog/schema/path, timezone, helper lineage, audit helper.  |
| 01_generate_or_upload_data | Mô tả generator local và path landing trên Databricks.            |
| 02_bronze_ingest           | Auto Loader/File ingest cho Bronze, append-only, metadata.        |
| 03_quarantine_rules        | DQ rules và ghi quarantine/error tables.                          |
| 04_silver_events           | Normalize event_time/device, dedup, partition, constraints.       |
| 05_silver_orders_items     | SCD1 orders, item versioning, FX conversion.                      |
| 06_silver_customers_scd2   | Customer SCD2, late/stale records.                                |
| 07_gold_marts              | 4 bảng Gold.                                                      |
| 08_delta_features          | CDF/watermark, time travel, optimize, vacuum, audit demo.         |
| 09_validation              | Queries chứng minh idempotency, DQ, row counts, business outputs. |

## 14. Checklist Nghiệm Thu

- Bronze có đủ ingest\_ts, source\_file, batch\_id, load\_date.
- Bronze append-only, không update/delete raw records.
- Có quarantine cho parse\_error, missing\_key, invalid\_timestamp, schema\_mismatch, duplicate\_key, fx\_rate\_missing.
- silver\_events không trùng event\_id và event\_time là timestamp UTC.
- silver\_orders chỉ giữ latest state mỗi order\_id.
- silver\_order\_items bảo toàn qty/price theo từng update hoặc giải thích rõ nếu overwrite.
- silver\_customers có effective\_from, effective\_to, is\_current và chỉ một current row mỗi user\_id.
- Gold đủ 4 bảng và chạy được dashboard query.
- Có MERGE cho silver\_orders và silver\_customers.
- Có demo Time Travel với VERSION AS OF hoặc TIMESTAMP AS OF.
- Có audit\_pipeline\_runs ghi rows\_read, rows\_written, rows\_quarantined, source\_paths.
- Có OPTIMIZE/ZORDER ít nhất hai bảng và số liệu small files trước/sau.
- Có VACUUM retention và giải thích tradeoff.
- Có CDF cho ít nhất một bảng Silver hoặc watermark lookback chứng minh late data.
- Chạy lại pipeline không làm tăng duplicate ở Silver/Gold.

## 15. Các Lỗi Dễ Mất Điểm

- Chỉ nói Bronze/Silver/Gold chung chung nhưng không có DQ/quarantine cụ thể.
- Không chứng minh idempotency khi chạy lại cùng batch.
- Đọc mixed JSON/Parquet bằng một loader duy nhất rồi fail ngầm.
- Bỏ qua schema drift device hoặc normalize device bằng string parsing quá hẹp.
- SCD2 customers chỉ update current mà không xử lý late record.
- Gold revenue không trừ cancelled/returned.
- Gold daily funnel bỏ anonymous user mà không nêu rule.
- Dùng VACUUM retention quá ngắn rồi vẫn claim time travel lâu dài.
- Không có audit lineage đến batch\_id/source\_file.

## 16. Cách Trình Bày Khi Bảo Vệ

- 1 Bắt đầu bằng kiến trúc: Landing -> Bronze -> Silver -> Gold, nêu vai trò từng layer.
- 2 Cho xem manifest/data generator để chứng minh dataset có lỗi và edge cases.
- 3 Chạy một batch hoặc show output Bronze có metadata lineage.
- 4 Show quarantine counts theo category.
- 5 Show Silver Events dedup và normalize device.
- 6 Show Silver Orders MERGE latest state và FX conversion.
- 7 Show Silver Customers SCD2 với một user có nhiều version.
- 8 Show 4 bảng Gold và một vài query business.
- 9 Show Time Travel, Optimize/ZORDER metrics, Vacuum policy.
- 10 Kết thúc bằng idempotency test: chạy lại không tăng duplicate.

Thông điệp chốt: bài này không chỉ là ETL. Điểm mạnh nằm ở việc pipeline có thể replay, audit, xử lý lỗi, xử lý dữ liệu đến trễ và tối ưu được trên Delta Lake.

## 17. Tài Liệu Databricks Chính Thức

- Unity Catalog volumes: khái niệm, path, compute requirements và limitations.
- Create and manage volumes: CREATE VOLUME, managed/external volume.
- Work with files in volumes: upload UI, programmatic access và CLI.
- Databricks CLI fs commands cho dbfs:/Volumes paths.
- Privileges for volumes: USE CATALOG, USE SCHEMA, READ VOLUME, WRITE VOLUME.